



FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433 - SOFTWARE ARCHITECTURE

SEMESTER II SESSION 25/26

BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING)

WITH HONOURS

**Microservice-Based Disaster Relief Coordination Platform Architecture Design
(Group 1)**

Submitted To:

Dr. Mohamad Nor Hasan

NAME	MATRIC NO.
SATRIALDY DARREL FARIZKI	S76351
FLORA TANG SUE QI	S74646
MOHAMED ASHRAFF BIN MOHAMED MUHRIZAN	S62368

1. System Overview

The Microservice-Based Disaster Relief Coordination Platform is designed to ensure swift, reliable, and highly available orchestration of emergency responses. Disasters strike unexpectedly, often disabling infrastructure and causing massive spikes in digital traffic as victims and responders attempt to coordinate. Traditional monolithic systems risk total failure under such sudden loads. This platform mitigates that risk by distributing responsibilities across independent, specialized microservices.

Driven by a React frontend and served via an Nginx web server, user requests are routed through a Kong API Gateway. This central gateway acts as the reverse proxy, handling rate limiting and routing to backend services built on Quarkus. Keycloak governs Identity and Access Management (IAM), ensuring that actions are strictly authorized via Role-Based Access Control (RBAC). Through this robust setup, the system provides real-time situational awareness, seamless volunteer deployment, and efficient resource allocation to disaster shelters.

The core of this new system revolves around a strict microservice decomposition: Volunteer Service, Resource Service, Shelter Service, and Alert Service. These components adhere to extreme decoupling principles, managing their own isolated data schemas to prevent cross-domain contamination and ensure maximum operational resilience.

2. Functional Requirement

The functional capabilities of the platform are meticulously categorized to serve the distinct needs of field operations and administrative oversight. The system guarantees that specific features are tightly bound to authenticated roles.

2.1 Identity Service

- **Centralized Security:** Manages all authentication, authorization, and user role operations via Keycloak.
- **Access Governance:** Issues and validates OAuth2/JWT tokens to secure internal microservices routed through the API gateway.
- **Role-Based Access Control (RBAC):** Enforces strict capability limits based on the authenticated persona (e.g., Victims, Responders, Administrators).
- **Responder Verification:** Mandates that field personnel submit official credentials to unlock elevated privileges for live incident updates and resource tracking.

2.2 Incident Service

- **Lifecycle Management:** Manages the end-to-end lifecycle of disaster reports and incoming field data.
- **Multimodal Reporting:** Enables Victims to submit SOS signals with geolocation tags and attached images of the disaster scene.
- **Responder & Admin Inputs:** Allows Responders to log newly discovered field incidents, while Administrators can manually input data received from external agencies.
- **Status Tracking & Resolution:** Facilitates real-time tracking, allowing Responders to update field conditions and Administrators to override records or formally close event tickets.

2.3 Resource Service

- **Inventory Management:** Manages the complete inventory, logistics, and stock updates for all relief supplies across the platform.

- **Automated Allocation:** Automatically evaluates available inventory schemas and deducts required relief supplies based on system workflow triggers or incoming requests.
- **Role-Specific Access:** Permits Responders to request essential supplies for active field operations, while granting Administrators full CRUD (Create, Read, Update, Delete) capabilities over logistics and supply chains.

2.4 GIS / Map Service

- **Geospatial Visualization:** Delivers comprehensive mapping and coordination tools, featuring heatmap generation, interactive mapping, and proximity-based routing.
- **Role-Specific Map Access:** Tailors map visibility by user persona: Victims can view safe zones and public shelters, Responders access supply depot locations and optimal routing, while Administrators monitor a comprehensive geospatial heatmap of all active operations.
- **Dynamic Tracking & Updates:** Acts as an event subscriber to dynamically calculate destination parameters and update active safe-zone and shelter metrics in response to field changes or resource allocations.

2.5 Notification Service

- **Real-Time Communication:** Facilitates instant communication, emergency alerts, and active monitoring broadcasts across the platform.
- **Alert Modalities:** Drives live chat via WebSockets, push notifications, and targeted geo-fenced emergency broadcasting.
- **High-Speed State Management:** Connects exclusively to a Redis in-memory cache to handle high-velocity, ephemeral data such as active field responder sessions and live alert states.

3. Non-Functional Requirement

- **High Availability & Fault Tolerance:** The system must maintain 99.9% uptime. Failure of a secondary service (like Alert Service) must degrade gracefully without bringing down critical transaction paths (like Volunteer Registration).
- **Independent Scalability:** Services must scale horizontally. During a mass casualty event, the Alert and Resource services must scale independently based on CPU and memory thresholds without scaling the Shelter configuration modules.
- **Data Security & Isolation:** All inter-service communications over REST must be secured. User endpoints must be validated via Keycloak JWTs. No service is permitted to read another service's database schema.
- **Performance & Latency:** Standard REST interactions must resolve in under 200ms. Asynchronous event propagation via RabbitMQ must be delivered to subscriber queues in under 50ms.
- **Maintainability:** Codebases must be physically separated by domain boundaries, packaged via Docker containers to ensure consistent local, staging, and production environments.

4. Architecture Style Explanation

The Microservice-Based Disaster Relief Coordination Platform (DRCP) is designed using a **Microservices Architecture Style**. This approach is selected to meet the critical demands of disaster management, where reliability and speed are non-negotiable.

4.1 Core Philosophy: Decoupling and Resilience

The fundamental principle of this style is the decomposition of the system into independent, specialized services.

- **Domain Isolation:** Responsibilities are split into distinct domains such as Volunteer, Resource, Shelter, and Alert services.
- **Operational Resilience:** This separation ensures that if a non-critical service fails, it does not bring down essential transaction paths, such as victim registration or SOS submissions.
- **Data Sovereignty:** Each microservice manages its own isolated data schema, which prevents "cross-domain contamination" and ensures that services do not have unauthorized access to each other's databases.

4.2 High Availability and Fault Tolerance

Because disasters cause massive spikes in digital traffic, the architecture is built for extreme stability.

- **Horizontal Scalability:** Individual services can be scaled independently. For instance, during a mass casualty event, the Alert and Resource services can be scaled to handle more load without needing to scale the entire platform.
- **Graceful Degradation:** The system is designed for 99.9% uptime, ensuring that service failures are handled gracefully to maintain the core functionality of the platform.

4.3 Integration and Security

Despite being independent, the services are unified through a central gateway and robust messaging.

- **Kong API Gateway:** Serves as the single entry point for the React frontend, handling request routing and rate limiting to protect the backend services.
- **Centralized Identity (IAM):** Security is governed by **Keycloak**, which enforces Role-Based Access Control (RBAC). This ensures that every action is authorized via OAuth2/JWT tokens before it reaches the internal microservices.
- **Asynchronous Communication:** To maintain performance, the system uses **RabbitMQ** for asynchronous event propagation, ensuring that standard REST interactions resolve in under 200ms while events reach subscribers in under 50ms.

4.4 Standardized Packaging

To ensure the system works identically in development and production, the entire codebase is physically separated by domain boundaries and packaged using **Docker containers**. This containerization ensures that the environment is consistent, making the platform highly maintainable and easy to deploy.

5. Service or Component Identification

This section identifies the core functional units of the Disaster Relief Coordination Platform. Each service is physically separated and manages its own isolated database to ensure operational resilience.

5.1 Entry & Security Layer

- Kong API Gateway: Acts as the "Traffic Controller." All user requests from the React frontend pass through this gateway first. It handles request routing, ensures no single user overloads the system (rate limiting), and directs traffic to the correct backend service.
- Identity Service (Keycloak): Acts as the "Security Guard." It governs all authentication and authorization using Role-Based Access Control (RBAC). It issues secure digital keys (JWT tokens) so the system knows if a user is a Victim, a Responder, or an Administrator.

5.2 Core Operational Services

- Incident Service: Acts as the "Reporting Hub." It manages the entire lifecycle of disaster reports. It allows Victims to send SOS signals with images and GPS locations, while Responders use it to update field conditions in real-time.
- Resource Service: Acts as the "Warehouse Manager." It tracks the inventory of all relief supplies. It automatically calculates and allocates supplies based on incoming requests from the field, ensuring responders have what they need.
- GIS / Map Service: Acts as the "Navigator." It provides visual heatmaps and interactive maps. It helps victims find safe zones and public shelters, while providing responders with the best routes to reach incident sites.

5.3 Communication & Messaging Layer

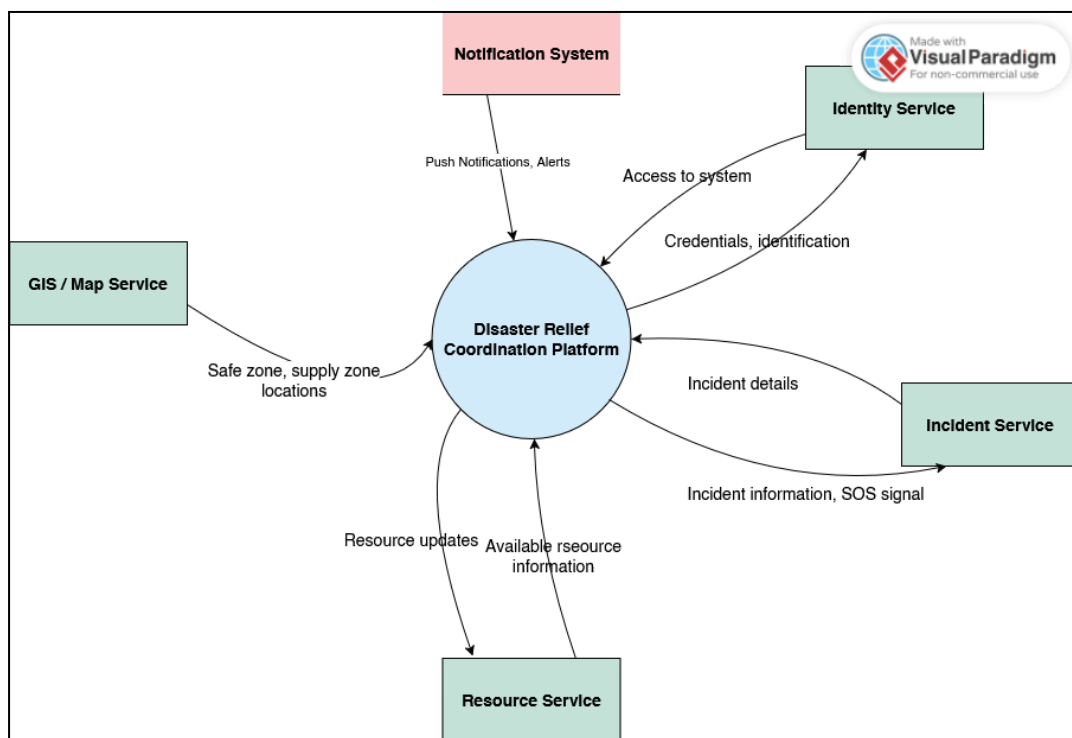
- Notification Service: Acts as the "Broadcaster." It facilitates instant communication, such as push notifications and emergency alerts.
- Redis (In-Memory Cache): Supports the Notification Service by handling "High-Speed Data." It stores information temporarily like live responder sessions and active alert states for near-instant access.
- RabbitMQ (Message Broker): Acts as the "Messenger" between services. It allows one

service (like the Incident Service) to send an event to another (like the Notification Service) asynchronously, ensuring the system remains fast and decoupled.

6. Architecture Diagram

The following diagrams give a clear and easy-to-understand view of how the Disaster Relief Coordination Platform (DRCP) is structured and how its components work together. They help visualize the flow of information and show how different services interact within the system. The Component/Service Architecture Diagram highlights the main services, like Incident and Resource Management, and how they communicate with each other through secure channels. Meanwhile, the Interaction Diagram walks you through the process of reporting an incident, from the user submitting an SOS signal to the system notifying responders. These diagrams offer a simplified yet insightful look into the platform's design, focusing on its efficiency, scalability, and ability to handle disaster scenarios with ease.

6.1 System Context Diagram

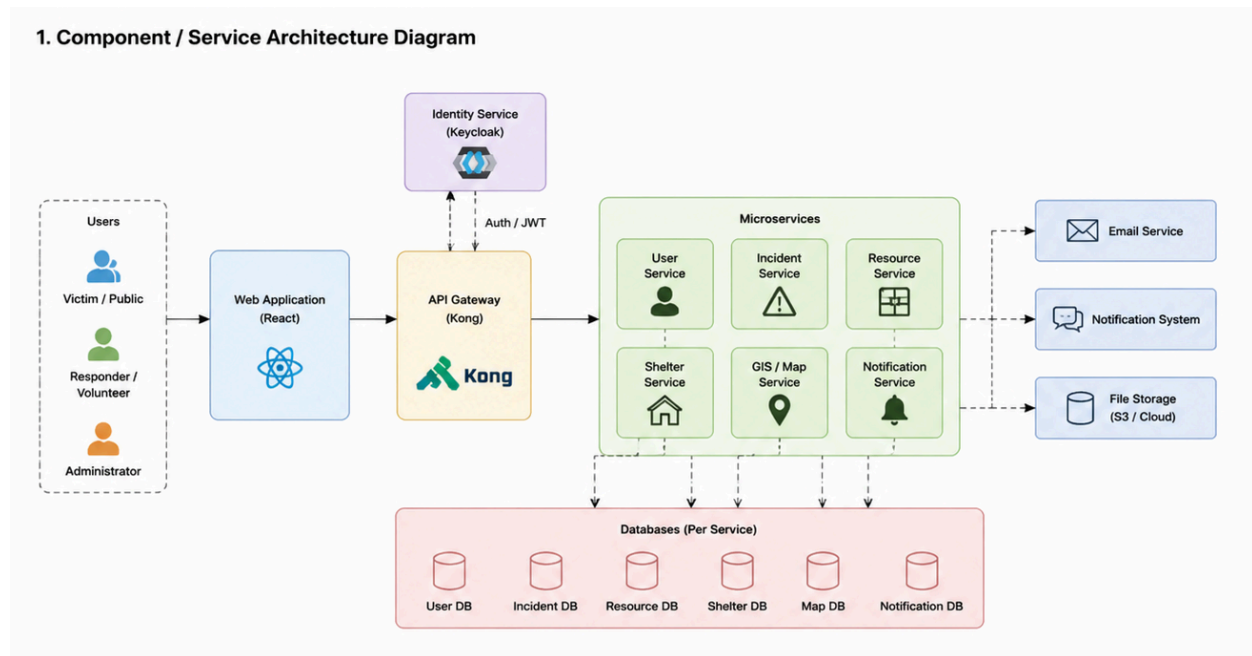


6.1 DRCP System Context Diagram

This image is a **System Context Diagram** that illustrates the high-level data flow and interactions between the central **Disaster Relief Coordination Platform** and its core independent services.

Key interactions depicted in the diagram include the services that have been provided in the service or component identification section.

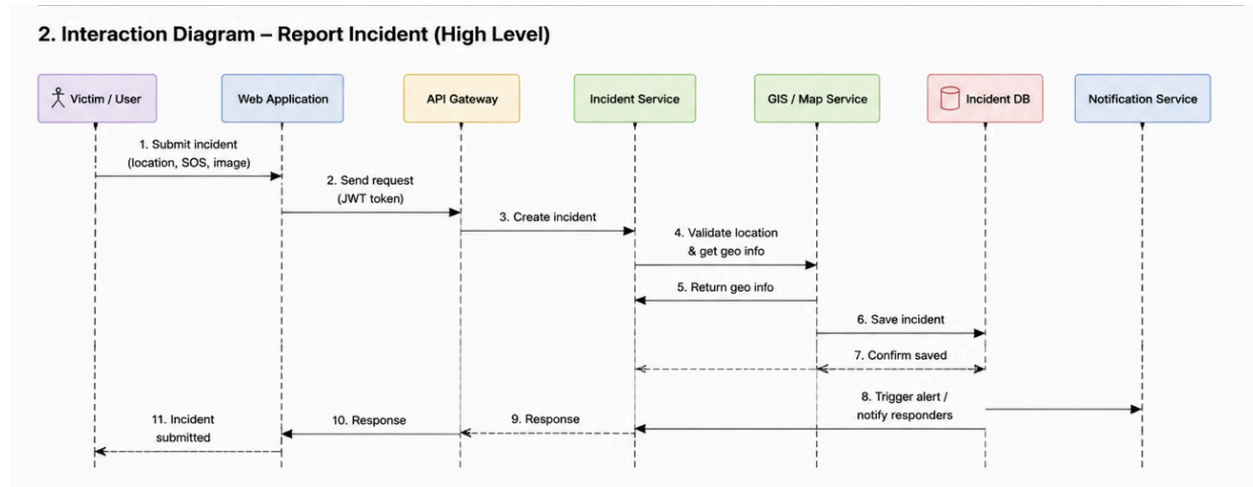
6.2 Service Architecture Diagram



6.2 Service Architecture Diagram

This diagram shows the structure of the Disaster Relief Coordination Platform (DRCP) and how its different parts work together. At the center, users whether they are Victims, Responders, or Administrators interact with the system through a web application. The Kong API Gateway is the main entry point, handling secure logins and permissions through Keycloak, which ensures that only authorized users can access certain features. The platform is built using different microservices that focus on specific tasks, like managing users, incidents, resources, shelters, maps, and notifications. Each of these services has its own dedicated database, keeping everything organized and running smoothly. To stay connected, the platform integrates external services like email and notifications for communication, and uses cloud storage (e.g., AWS S3) to handle media like images from users. This setup makes sure the system is reliable, scalable, and secure,

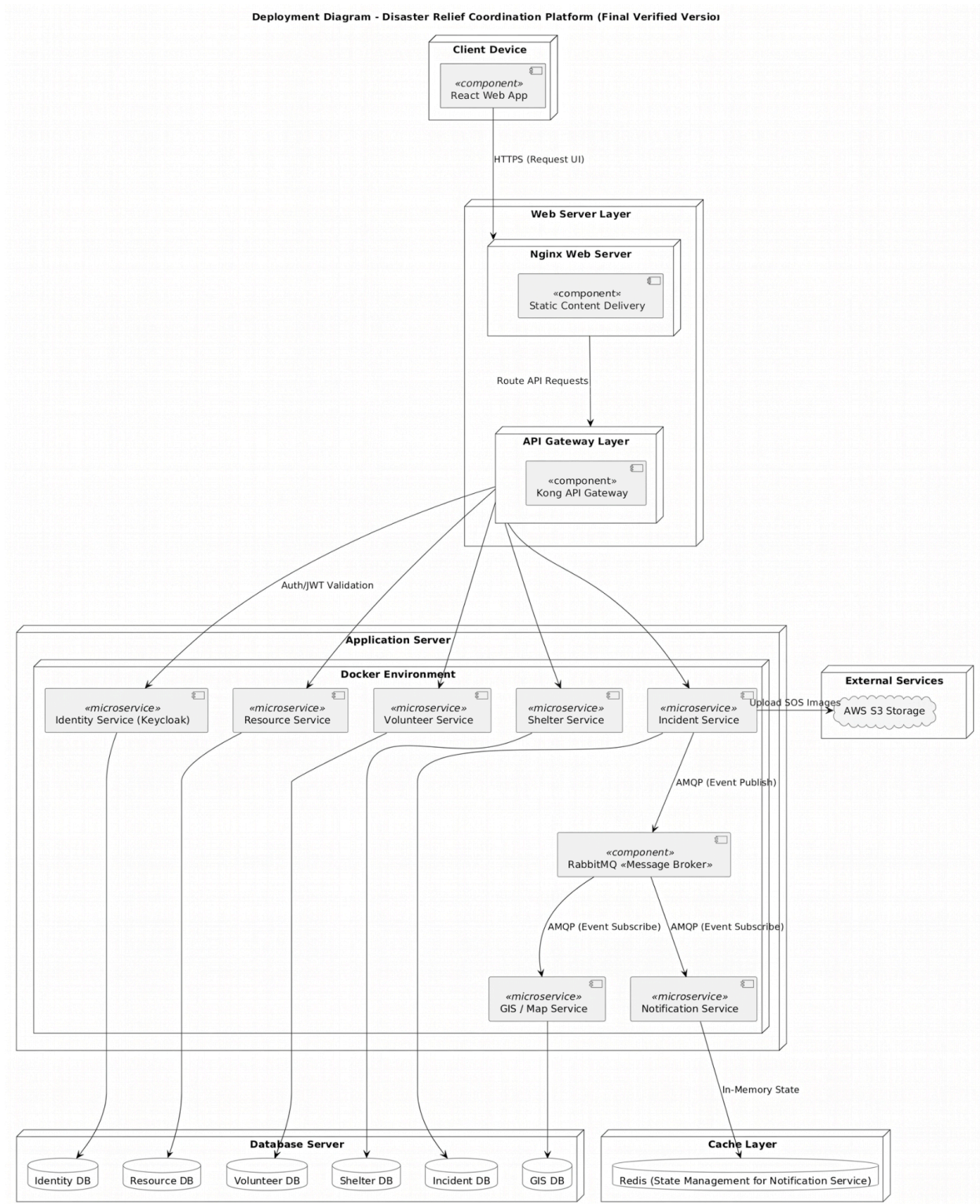
6.3 Interaction Diagram



6.3 Interaction Diagram

The Interaction Diagram explains the steps taken when a Victim/User reports an incident through the platform. It all starts with the user submitting a report that includes important details like their location, an SOS signal, and any images they can provide. Once the report is submitted, it's securely verified using JWT tokens and sent through the API Gateway. From there, the Incident Service takes over, processing the information and creating an official record of the incident. The GIS/Map Service checks the location and pulls up relevant geospatial data, which is sent back to the Incident Service. The incident details are then stored in the Incident Database, and the platform sends out alerts through the Notification Service to notify responders. Finally, the user gets a confirmation that their incident has been recorded, allowing for quick follow-up and action from the authorities. This diagram gives a clear, step-by-step view of how incidents are reported and tracked, ensuring a fast and organized response in times of disaster.

6.4 Deployment Diagram



6.4 Deployment Diagram

This Deployment Diagram for the Disaster Relief Coordination Platform illustrates a tiered architecture that manages data flow from users to secure storage. The process begins with a user on a Client Device (React Web App) connecting to a central Web Server Layer (Nginx and Kong API Gateway), which handles initial requests, security, and traffic routing. The requests are then passed to the core Application Server, which uses a Docker Environment to host isolated, containerized microservices like Incident, Volunteer, and Map services. These services coordinate in real-time, sharing information like map updates and alerts via a RabbitMQ Message Broker, and storing persistent data, such as resource inventory and volunteer profiles, across a dedicated Database Server. Finally, the system integrates with external cloud services (AWS S3) for secure photo uploads and utilizes a fast in-memory Cache Layer (Redis) to ensure rapid notification delivery.